

Tracky

Personal Data Dashboard

Phase 1 — Project Documentation

1. Author Information

Field	Value
Name	Amir Abdullah Zakaria
Neptun Code	ZD0LBW
Course	Advance Web Programming
Project	Tracky — Personal Data Dashboard

2. Project Overview

2.1 Topic and Short Description

Tracky is a personal data dashboard application that lets users define their own metrics, log daily values, and visualize trends through a fully configurable widget-based dashboard. It targets individuals who want a single, private, flexible tool to track health, productivity, or any numeric life data — without being locked into a fixed set of predefined categories.

Users create metrics of three types (numeric, scale/rating, boolean), record entries each day, and arrange visual widgets on one or more named dashboards. The app solves the problem of scattered tracking tools by providing a unified, highly customizable personal analytics layer.

2.2 Domain Vocabulary

Term	Definition
Metric	A named measurement the user chooses to track (e.g. "Sleep Hours", "Mood", "Exercise").
Entry / Log	A single recorded value for a metric on a specific date.
Dashboard	A named collection of widgets the user curates (e.g. "Health", "Productivity").
Widget	A visual card placed on a dashboard, bound to one metric, displaying it as a chosen chart type.
Chart Type	The visual representation of a widget: Line Chart, Bar Chart, Stat Card, or Streak.
Streak	A count of consecutive days on which a boolean metric evaluated to true (or a numeric metric exceeded a threshold).
Metric Type	One of three data flavours: numeric (float), scale (integer with min/max), boolean (true/false).
Widget Config	JSON blob on the widget row storing chart-type-specific settings such as date range or comparison mode.

3. Technology Stack

3.1 Server-Side

Technology	Role	Why chosen
Laravel (REST API)	HTTP layer, routing, business logic, data access	Mature PHP framework with first-class REST support, expressive ORM (Eloquent), and built-in validation.
Laravel Breeze + Sanctum	Authentication scaffolding and SPA token issuance	Sanctum issues cookie-based or token-based sessions for the SPA with minimal configuration; Breeze adds ready-made auth controllers.
SQLite	Relational database engine	Zero-config, file-based database suitable for a personal project; no separate DB server needed.

3.2 Client-Side

Technology	Role	Why chosen
React	UI component framework	Component model maps cleanly to dashboard widgets and the multi-step widget configurator; large ecosystem for charting libraries.
TanStack Query	Server state management and data fetching	Handles caching, background refetching, and mutation side-effects (cache invalidation after log entry save) with minimal boilerplate.
Tailwind CSS	Utility-first styling	Rapid responsive layout without leaving JSX; pairs well with Flowbite components.
Flowbite	Component library	Pre-built accessible components (modals, dropdowns, toggles, cards) built on Tailwind — consistent look-and-feel without a heavy CSS framework.
Recharts / Chart.js	Data visualisation	Declarative chart components rendered client-side; easily driven by the entry data returned from the API.

4. Database Schema

4.1 Entity List and Responsibilities

Table	Responsibility
users	Stores authentication credentials and profile data. Acts as the root owner of all other domain entities.
metrics	Defines a named measurable quantity owned by a user. Stores the metric type (numeric / scale / boolean) and type-specific config (unit, min, max).
entries	Records a single logged value for a metric on a specific date. This is the core 1:N child table: one metric has many entries.
dashboards	A named grouping of widgets owned by a user. A user may have many dashboards.
widgets	A widget placed on a dashboard. References one metric and stores the chosen chart type plus a JSON config blob with chart-specific settings.

4.2 Fields Per Table

users

Column	Type	Null / Default	Description
id	bigint (PK, AI)	NOT NULL	Primary key.
name	string(255)	NOT NULL	Display name of the user.
email	string(255)	NOT NULL, UNIQUE	Login email address.
email_verified_at	datetime	NULL	Timestamp of email verification; null if unverified.
password	string(255)	NOT NULL	Bcrypt-hashed password.
role	enum('user','admin')	DEFAULT 'user'	Authorization role. Admins have read access to all data.
remember_token	string(100)	NULL	Laravel auto-generated remember-me token.
created_at	datetime	NOT NULL	Row creation timestamp.
updated_at	datetime	NOT NULL	Row update timestamp.

metrics

Column	Type	Null / Default	Description
id	bigint (PK, AI)	NOT NULL	Primary key.
user_id	bigint (FK → users.id)	NOT NULL	Owner. Cascade delete.
name	string(100)	NOT NULL	Human-readable name (e.g. 'Sleep Hours').

description	text	NULL	Optional longer description or goal note.
type	enum('numeric','scale','boolean')	NOT NULL	Determines the input widget and valid entry values.
unit	string(30)	NULL	Unit label for numeric metrics (e.g. 'kg', 'hours'). NULL for scale/boolean.
scale_min	integer	NULL, DEFAULT 1	Minimum value for scale metrics.
scale_max	integer	NULL, DEFAULT 10	Maximum value for scale metrics.
is_active	boolean	NOT NULL, DEFAULT true	Whether the metric appears on the log entry page.
created_at	datetime	NOT NULL	Row creation timestamp.
updated_at	datetime	NOT NULL	Row update timestamp.

entries

Column	Type	Null / Default	Description
id	bigint (PK, AI)	NOT NULL	Primary key.
metric_id	bigint (FK → metrics.id)	NOT NULL	The metric being logged. Cascade delete.
user_id	bigint (FK → users.id)	NOT NULL	Redundant owner reference for fast query scoping without join.
logged_date	date	NOT NULL	The date the value represents (not necessarily today).
value_numeric	decimal(10,2)	NULL	Populated for numeric metric types.
value_scale	integer	NULL	Populated for scale metric types.
value_boolean	boolean	NULL	Populated for boolean metric types.
notes	text	NULL	Optional free-text note for context.
created_at	datetime	NOT NULL	Row creation timestamp.
updated_at	datetime	NOT NULL	Row update timestamp.

UNIQUE constraint on (metric_id, logged_date): a user may log a metric at most once per day.

dashboards

Column	Type	Null / Default	Description
id	bigint (PK, AI)	NOT NULL	Primary key.
user_id	bigint (FK → users.id)	NOT NULL	Owner. Cascade delete.
name	string(100)	NOT NULL	Dashboard name (e.g. 'Health', 'Productivity').
description	text	NULL	Optional description of the dashboard's purpose.
created_at	datetime	NOT NULL	Row creation timestamp.
updated_at	datetime	NOT NULL	Row update timestamp.

widgets

Column	Type	Null / Default	Description
id	bigint (PK, AI)	NOT NULL	Primary key.
dashboard_id	bigint (FK → dashboards.id)	NOT NULL	Parent dashboard. Cascade delete.
metric_id	bigint (FK → metrics.id)	NOT NULL	The metric this widget visualises.
chart_type	enum('line','bar','stat','streak')	NOT NULL	Chosen visualisation type.
position	integer	NOT NULL, DEFAULT 0	Display order on the dashboard grid; used for drag-and-drop reordering.
config	JSON	NOT NULL, DEFAULT '{}'	Chart-type-specific settings: date range, comparison mode, threshold, colour, etc.
created_at	datetime	NOT NULL	Row creation timestamp.
updated_at	datetime	NOT NULL	Row update timestamp.

4.3 Relationships

Each User owns many Metrics (User 1 → N Metrics). A user's metrics are entirely private; no metric is shared across users.

Each Metric has many Entries (Metric 1 → N Entries). This is the most heavily used relationship in the application: every chart query fetches entries for a metric within a date range. The `user_id` is denormalized onto entries for fast scoping.

Each User owns many Dashboards (User 1 → N Dashboards). A user may organise their widgets into named themed dashboards.

Each Dashboard has many Widgets (Dashboard 1 → N Widgets). Each Widget is bound to exactly one Metric (Metric 1 → N Widgets, across all dashboards).

4.4 Entity-Relationship Diagram (Textual)

Node	Edges (relationships)
users	1 → N metrics 1 → N dashboards 1 → N entries (via user_id)
metrics	N → 1 users 1 → N entries 1 → N widgets
entries	N → 1 metrics N → 1 users (denormalized)
dashboards	N → 1 users 1 → N widgets
widgets	N → 1 dashboards N → 1 metrics

4.5 DBML

```

Table users {
  id bigint [pk, increment]
  name varchar(255) [not null]
  email varchar(255) [not null, unique]
  email_verified_at datetime
  password varchar(255) [not null]
  role varchar(10) [not null, default: 'user']
  remember_token varchar(100)
  created_at datetime [not null]
  updated_at datetime [not null]
}

Table metrics {
  id bigint [pk, increment]
  user_id bigint [not null, ref: > users.id]
  name varchar(100) [not null]
  description text
  type varchar(10) [not null, note: 'numeric|scale|boolean']
  unit varchar(30)
  scale_min int [default: 1]
  scale_max int [default: 10]
  is_active boolean [not null, default: true]
  created_at datetime [not null]
  updated_at datetime [not null]
}

Table entries {
  id bigint [pk, increment]
  metric_id bigint [not null, ref: > metrics.id]
  user_id bigint [not null, ref: > users.id]
  logged_date date [not null]
  value_numeric decimal(10,2)
  value_scale int
  value_boolean boolean
  notes text
  created_at datetime [not null]
  updated_at datetime [not null]

  indexes {
    (metric_id, logged_date) [unique]
  }
}

Table dashboards {

```

```
id bigint [pk, increment]
user_id bigint [not null, ref: > users.id]
name varchar(100) [not null]
description text
created_at datetime [not null]
updated_at datetime [not null]
}

Table widgets {
  id bigint [pk, increment]
  dashboard_id bigint [not null, ref: > dashboards.id]
  metric_id bigint [not null, ref: > metrics.id]
  chart_type varchar(10) [not null, note: 'line|bar|stat|streak']
  position int [not null, default: 0]
  config json [not null, default: '{}']
  created_at datetime [not null]
  updated_at datetime [not null]
}
```

5. API Routes Plan (REST)

All authenticated routes require a valid Sanctum Bearer token. Owner-scoped resources enforce ownership in a Laravel Policy; admin role bypasses ownership checks. All routes are prefixed /api.

5.1 Authentication

Method	Path	Function	Auth	Authorization
POST	/api/register	Register a new user account	Public	Anyone
POST	/api/login	Authenticate and return a Sanctum token	Public	Anyone
POST	/api/logout	Revoke the current token	Authenticated	Authenticated user
GET	/api/user	Return the authenticated user's profile	Authenticated	Authenticated user
PATCH	/api/user	Update profile (name, email, password)	Authenticated	Authenticated user

5.2 Metrics (Owner-scoped, full CRUD)

All metric routes are owner-scoped: a user may only read or modify metrics where `metrics.user_id = auth()->id()`. Admins may read any metric but cannot modify another user's metric.

Method	Path	Function	Auth	Authorization
GET	/api/metrics	List all metrics for the authenticated user	Authenticated	Owner
POST	/api/metrics	Create a new metric for the authenticated user	Authenticated	Owner
GET	/api/metrics/{id}	Get a single metric (with type config)	Authenticated	Owner or Admin
PATCH	/api/metrics/{id}	Update metric name, description, unit, scale bounds	Authenticated	Owner only
DELETE	/api/metrics/{id}	Delete a metric and cascade its entries and widgets	Authenticated	Owner only

5.3 Entries (Owner-scoped, full CRUD)

Entries are doubly scoped: the resolved metric must belong to the authenticated user. The PATCH `/api/entries/bulk` endpoint accepts an array of `{metric_id, date, value}` objects for the Quick Log page.

Method	Path	Function	Auth	Authorization
GET	/api/entries	List entries; filterable by	Authenticated	Owner

		metric_id, date range, and page		
POST	/api/entries	Create a single entry for a metric	Authenticated	Owner
POST	/api/entries/bulk	Upsert multiple entries in one request (Quick Log submit)	Authenticated	Owner
GET	/api/entries/{id}	Get a single entry	Authenticated	Owner or Admin
PATCH	/api/entries/{id}	Update a logged value or notes	Authenticated	Owner only
DELETE	/api/entries/{id}	Delete a single entry	Authenticated	Owner only
GET	/api/metrics/{id}/entries	Get paginated entries for a specific metric (chart data source)	Authenticated	Owner or Admin

5.4 Dashboards (Owner-scoped, full CRUD)

Method	Path	Function	Auth	Authorization
GET	/api/dashboards	List all dashboards for the authenticated user	Authenticated	Owner
POST	/api/dashboards	Create a new dashboard	Authenticated	Owner
GET	/api/dashboards/{id}	Get dashboard with its widgets (eager-loaded)	Authenticated	Owner or Admin
PATCH	/api/dashboards/{id}	Update dashboard name or description	Authenticated	Owner only
DELETE	/api/dashboards/{id}	Delete dashboard and all its widgets	Authenticated	Owner only

5.5 Widgets (Owner-scoped, full CRUD)

The PATCH `/api/dashboards/{id}/widgets/reorder` endpoint accepts an ordered array of widget IDs and updates the position field of each. This is called after a drag-and-drop reorder.

Method	Path	Function	Auth	Authorization
GET	/api/dashboards/{id}/widgets	List widgets on a dashboard (ordered by position)	Authenticated	Owner

POST	/api/dashboards/{id}/widgets	Add a new widget to the dashboard	Authenticated	Owner
GET	/api/widgets/{id}	Get a single widget with its config	Authenticated	Owner or Admin
PATCH	/api/widgets/{id}	Update chart_type or config of a widget	Authenticated	Owner only
DELETE	/api/widgets/{id}	Remove a widget from the dashboard	Authenticated	Owner only
PATCH	/api/dashboards/{id}/widgets/reorder	Reorder widgets by submitting new position array	Authenticated	Owner only

5.6 Admin Routes

Method	Path	Function	Auth	Authorization
GET	/api/admin/users	List all users with summary stats	Authenticated	Admin only
GET	/api/admin/users/{id}	Get full profile and metrics of any user	Authenticated	Admin only
PATCH	/api/admin/users/{id}	Change a user's role	Authenticated	Admin only
DELETE	/api/admin/users/{id}	Hard-delete a user and all their data	Authenticated	Admin only

6. Client-Side Application Design

6.1 Pages and Navigation

Page	Purpose	Main Components	User Actions
Login / Register	Auth entry point	Flowbite form, validation errors	Submit credentials, link to register
Dashboard View	Main screen — shows one named dashboard's widgets	Widget grid, stat cards, charts, drag handle, Add Widget button	View charts, drag-reorder, open configurator, delete widget
Dashboard List	Manage all dashboards	Card list, create/rename/delete controls	Create new dashboard, navigate to dashboard, delete
Metrics	CRUD for personal metrics	Table with type badges, edit/delete buttons	Create metric (dynamic form), edit, toggle active, delete
Log Entry	Quick log today's values	Per-metric input cards (boolean toggle, scale dots, numeric input), Save All button	Enter values, save individually or all at once
Entry History	Browse past log entries for a metric	Date-filtered table, inline edit	Filter by date, edit value, delete entry
Profile / Settings	Account management	Flowbite form	Update name, email, password
Admin Panel	User management (admin only)	User table, role badge, stats	View all users, change role, delete user

6.2 Non-Trivial Client-Side Features

Feature A — Widget Configurator (multi-step form with dependent fields)

Location: Dashboard View page, triggered by the '+ Add Widget' button or the edit (✎) icon on an existing widget.

Step 1 — Select Metric: A searchable list of the user's metrics is shown. Selecting a metric reveals its type badge (numeric / scale / boolean). The selected metric_id is held in local state.

Step 2 — Choose Chart Type: Four chart type cards are displayed. Available types are filtered based on the metric type: boolean metrics only show 'Streak' and 'Stat Card'; numeric and scale metrics show all four. Selecting a chart type updates local state and advances to step 3.

Step 3 — Configure (dependent fields): This step renders different fields based on the chart type chosen in step 2:

- Line Chart: date range selector (7d / 14d / 30d / 90d), show data points toggle, line colour picker.
- Bar Chart: grouping period (daily / weekly / monthly), bar colour picker.
- Stat Card: comparison mode dropdown (today vs average / today vs yesterday / today vs last week).

- Streak: threshold type (boolean = true, or numeric >= value), threshold value input if numeric.

Local React state tracks: `currentStep` (1|2|3), `selectedMetricId`, `selectedChartType`, `configDraft` (object). The config object is type-narrowed on chart type change — switching chart type resets `configDraft` to the defaults for the new type.

On submit, a POST `/api/dashboards/{id}/widgets` (or PATCH `/api/widgets/{id}` for edit) is called via a TanStack Query mutation. On success, the widgets query for the current dashboard is invalidated, causing the grid to re-fetch and show the new widget.

Feature B — Drag-and-Drop Widget Reordering

Location: Dashboard View page — all widgets on the grid have a drag handle (⋮) visible on hover.

Implementation: The widget list is managed in local React state as an ordered array. On drag-end, the array is reordered immediately (optimistic update). A debounced call to PATCH `/api/dashboards/{id}/widgets/reorder` sends the new ordered array of widget IDs to the backend, which updates the position column. If the request fails, the local order is rolled back to the server-confirmed order.

This requires real UI state work: maintaining drag state, computing insertion index, rendering a placeholder drop zone, and coordinating the optimistic update with TanStack Query cache management.

6.3 React and TanStack Query Usage

Concern	Approach
Query keys	Hierarchical: <code>['metrics']</code> , <code>['metrics', id]</code> , <code>['dashboards']</code> , <code>['dashboards', id, 'widgets']</code> , <code>['entries', {metricId, from, to}]</code>
Mutations	<code>useMutation</code> with <code>onSuccess</code> calling <code>queryClient.invalidateQueries</code> for the affected resource list key.
Drag reorder	Optimistic update via <code>queryClient.setQueryData</code> before the API call; rollback in <code>onError</code> .
Auth token	Sanctum cookie-based auth for SPA. Axios interceptor reads the CSRF cookie; 401 responses redirect to <code>/login</code> .
401 handling	Axios response interceptor clears local auth state and navigates to <code>/login</code> .
Stale time	Metrics and dashboards: 5 minutes (rarely change). Entries: 0 (always fresh on focus).

7. Functional Requirements

7.1 High-Level Goals

- Users can register, log in, and manage their profile securely.
- Users can define personal metrics of three types: numeric (with unit), scale (with min/max), and boolean.
- Users can log one value per metric per day through the Quick Log page; values can be edited or deleted afterward.
- Users can create multiple named dashboards and populate each with widgets.
- Each widget binds to one metric and renders it as a line chart, bar chart, stat card, or streak counter.
- Widget chart type and configuration options are set through a multi-step configurator where available fields depend on the chosen chart type.
- Users can reorder widgets within a dashboard by dragging and dropping.
- The dashboard updates in real time as new entries are logged; no manual refresh is needed.
- Admins can view, manage, and delete any user and their data.
- All private data is strictly owner-scoped on the API: no user can read or modify another user's metrics, entries, dashboards, or widgets.

7.2 Features by Role

Capability	User	Admin
Register and log in	✓	✓ (pre-assigned)
Create / edit / delete own metrics	✓	X (for others)
Log entries for own metrics	✓	X (for others)
Create / manage own dashboards	✓	X (for others)
Add / configure / reorder widgets	✓	X (for others)
View own entry history	✓	✓ (any user)
View all users and their stats	X	✓
Change a user's role	X	✓
Delete any user and their data	X	✓

7.3 Features by Role and Page

Page	User	Admin
Login / Register	Full access	Full access
Dashboard View	View, add/edit/delete/reorder own widgets	Read-only view of any user's dashboard

Dashboard List	Create, rename, delete own dashboards	List own dashboards; can browse any user's via admin panel
Metrics	Full CRUD on own metrics; toggle active state	Read-only view of any metric
Log Entry	Log and edit values for own metrics	No access to other users' log entry page
Entry History	View, edit, delete own entries; filter by date	View any user's entries (read-only)
Profile / Settings	Update own name, email, password	Update own profile; change any user's role via admin panel
Admin Panel	No access (redirected)	Full access: user list, role management, delete user

8. Roles and Permissions

8.1 Roles

Role	Description
user	Default role assigned on registration. Full CRUD on their own resources (metrics, entries, dashboards, widgets). No visibility into other users' data.
admin	Elevated role assigned manually. Can read all users' data and manage user accounts (role changes, deletion). Cannot modify another user's metrics or entries — data integrity stays with the owner.

8.2 Permission Matrix

Resource	Action	Public	Authenticated User (own)	Authenticated User (other)	Admin
metrics	Create	X	✓	X	X
metrics	Read	X	✓	X	✓
metrics	Update	X	✓	X	X
metrics	Delete	X	✓	X	X
entries	Create	X	✓	X	X
entries	Read	X	✓	X	✓
entries	Update	X	✓	X	X
entries	Delete	X	✓	X	X
dashboards	Create	X	✓	X	X
dashboards	Read	X	✓	X	✓
dashboards	Update	X	✓	X	X
dashboards	Delete	X	✓	X	X
widgets	Create	X	✓	X	X
widgets	Read	X	✓	X	✓
widgets	Update	X	✓	X	X
widgets	Delete	X	✓	X	X
users	Read all	X	X	X	✓
users	Update role	X	X	X	✓
users	Delete	X	X	X	✓

8.3 Backend Enforcement

Middleware: The `auth:sanctum` middleware guards all API routes except `/api/register` and `/api/login`. An additional `admin` middleware checks `auth()->user()->role === 'admin'` for all `/api/admin/*` routes.

Policies: A `MetricPolicy`, `EntryPolicy`, `DashboardPolicy`, and `WidgetPolicy` each implement `view()`, `create()`, `update()`, and `delete()` methods. Each policy's `view()` and `update()` checks that the model's `user_id` equals `auth()->id()`, or the user is an admin where permitted.

Scoped queries: All index endpoints (GET `/api/metrics`, GET `/api/entries`, etc.) filter by `user_id = auth()->id()` at the query level — not just by hiding results in the response. This ensures that even crafted requests cannot leak cross-user data.

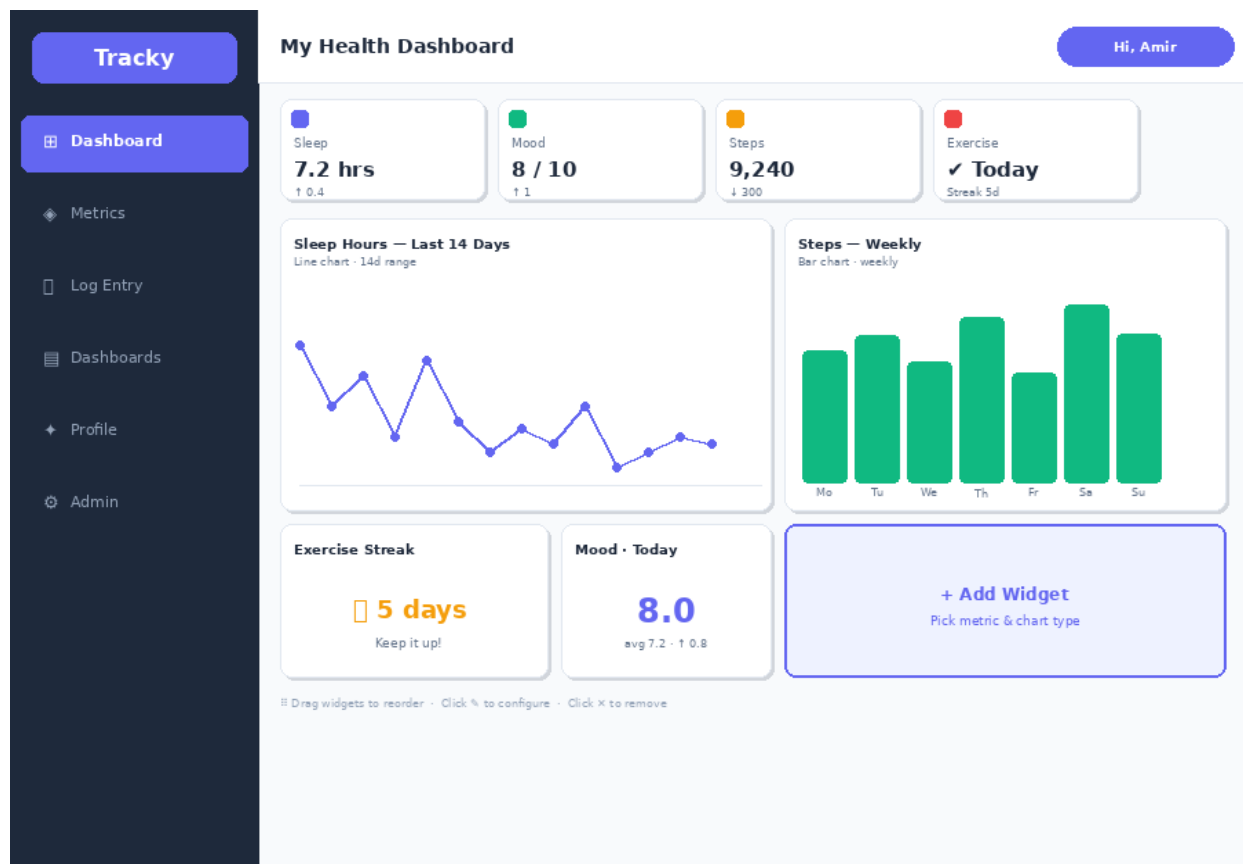
No UI-only protection: Role checks and ownership scoping are enforced exclusively in the backend. Frontend route guards (React Router) are a UX convenience only and are never relied upon for security.

9. UI Mockup Images

9.1 Dashboard View

Top row: 4 stat cards (Sleep, Mood, Steps, Exercise) showing today's value and a delta vs average. Below: a 2-column responsive grid of chart widgets — line chart for Sleep on the left, bar chart for Steps on the right. Below that: a Streak card for Exercise, a Mood Stat Card, and an '+ Add Widget' placeholder. Drag handle (::) appears on hover; edit (✎) and remove (✕) icons appear on each card corner.

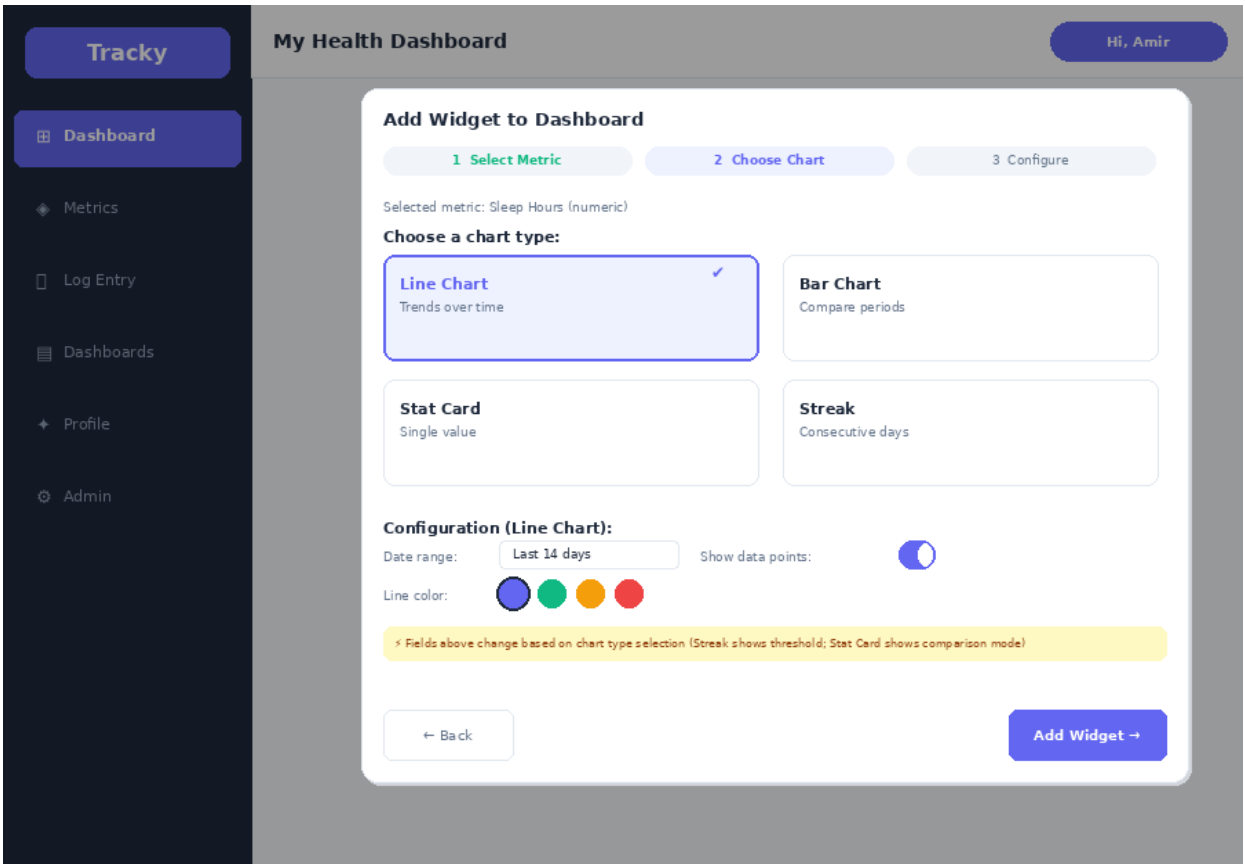
Mobile: sidebar collapses to a bottom tab bar; widget grid goes to a single column; stat cards wrap into a 2×2 grid.



9.2 Widget Configurator (Multi-Step Modal)

A centred modal overlaying the dashboard. A 3-step progress bar at the top (Select Metric → Choose Chart → Configure). Step 2 shown: four chart-type cards in a 2×2 grid; selected card highlighted in indigo with a checkmark. Step 3 below: dependent config fields rendered for the selected chart type (Line Chart shows date range dropdown, show-points toggle, colour swatches). A note banner explains that fields change with chart type. Back and 'Add Widget' buttons at the bottom.

Mobile: modal takes full screen; steps shown as a compressed horizontal bar.



9.3 Metrics Management

Full-width table listing all user metrics. Each row has a coloured left accent strip indicating type, a type badge (Numeric / Scale / Boolean), unit/range info, last entry summary, and edit/delete action buttons. A '+ New Metric' button in the top-right opens a dynamic creation form (type selection changes which additional fields appear: unit for numeric, min/max for scale, no extra fields for boolean).

Mobile: table collapses to stacked cards; action buttons move to a dropdown menu.

Tracky

Dashboard

Metrics

Log Entry

Dashboards

Profile

Admin

My Metrics

Hi, Amir

+ New Metric

Name	Type	Unit / Range	Last Entry	Actions
Sleep Hours	Numeric	hours	7.2 hrs · Yesterday	
Mood	Scale	1-10	8 · Today	
Weight	Numeric	kg	72.4 kg · 2d ago	
Exercise	Boolean	yes/no	✓ Today	
Energy	Scale	1-5	4 · Today	
Steps	Numeric	steps	9,240 · Today	

9.4 Log Entry (Quick Log)

Grid of per-metric input cards (2-column desktop, 1-column mobile). Each card shows the metric name, type badge, and an appropriate input: a toggle (YES/NO) for boolean, a 1–10 dot selector for scale, and a numeric text input with unit label for numeric. A 7-day average hint is shown below numeric inputs. Each card has an individual Save button; a global 'Save All Entries for Today' bar appears at the bottom.

Tracky

Dashboard

Metrics

Log Entry

Dashboards

Profile

Admin

Log Today's Data

Hi, Amir

Quick Log — May 4, 2026

Log values for all your active metrics

Sleep Hours

7.5

hours

7d avg: 7.8 hrs

Save

Mood

1

2

3

4

5

6

7

8

9

10

Selected: 8

Save

Exercise

YES

NO

Save

Steps

9500

steps

7d avg: 7.8 hrs

Save

✓ Save All Entries for Today

9.5 Admin Panel

Summary stat cards at the top (Total Users, Active Today, Metrics Defined, Entries Today). Below: a paginated user table with columns for Name, Email, Role (badge), Metrics count, Last Active, and a Ban/Delete action button. Role badges are colour-coded (indigo for Admin, green for User). Clicking a user row navigates to a read-only view of their data.

Mobile: table scrolls horizontally; admin panel is not designed for mobile-first use.

Tracky

Dashboard

Metrics

Log Entry

Dashboards

Profile

Admin

Admin Panel

Hi, Admin

User Management

Total Users

1,204

Active Today

87

Metrics Defined

3,891

Entries Today

5,420

User	Email	Role	Metrics	Last Active	Actions
Amir Z.	amin@ex.com		5	Today	Ben
Sara K.	sara@ex.com		12	Yesterday	Ben
John D.	john@ex.com		3	3d ago	Ben
Lena M.	lena@ex.com		8	Today	Ben
Max B.	max@ex.com		1	1w ago	Ben

10. AI Appendix

10.1 How AI Was Used in This Project

This Phase 1 documentation was produced with the assistance of Claude (Anthropic), used as an interactive design partner throughout the planning stage. The following tasks involved AI assistance:

- Brainstorming and evaluating project ideas against the assignment requirements (non-trivial UI, full CRUD, owner-scoped API).
- Iteratively refining the domain model — entity names, field choices, the decision to denormalise `user_id` onto entries, and the JSON config approach for widgets.
- Drafting the full API routes table, permission matrix, and functional requirements.
- Structuring the multi-step widget configurator feature so it clearly justifies non-trivial client-side state work.
- Generating wireframe mockup images programmatically (Python / Pillow) and producing this Word document (Node.js / docx).

All final decisions about scope, entity design, and feature priorities were made by the student. AI was used to accelerate drafting and surface consistency issues, not to replace design thinking.

10.2 Planned AI Usage in the Product

No AI behaviour is planned inside the Tracky application itself. The product is intentionally a data-tracking and visualisation tool driven entirely by user-logged values. There are no AI-generated suggestions, anomaly detection, or natural-language features planned for this project phase.

If the project is extended in a future phase, potential AI additions could include: automatic streak analysis commentary, anomaly flagging (e.g. 'Your sleep has dropped 20% this week'), or natural-language metric entry ('I slept 7 hours'). These are out of scope for Phase 1.